

# Memory Technologies

- Semiconductor

Registers,SRAM,DRAM,FLASH {all Random Access}

- Magnetic {all Random +sequential}

FDD,HDD

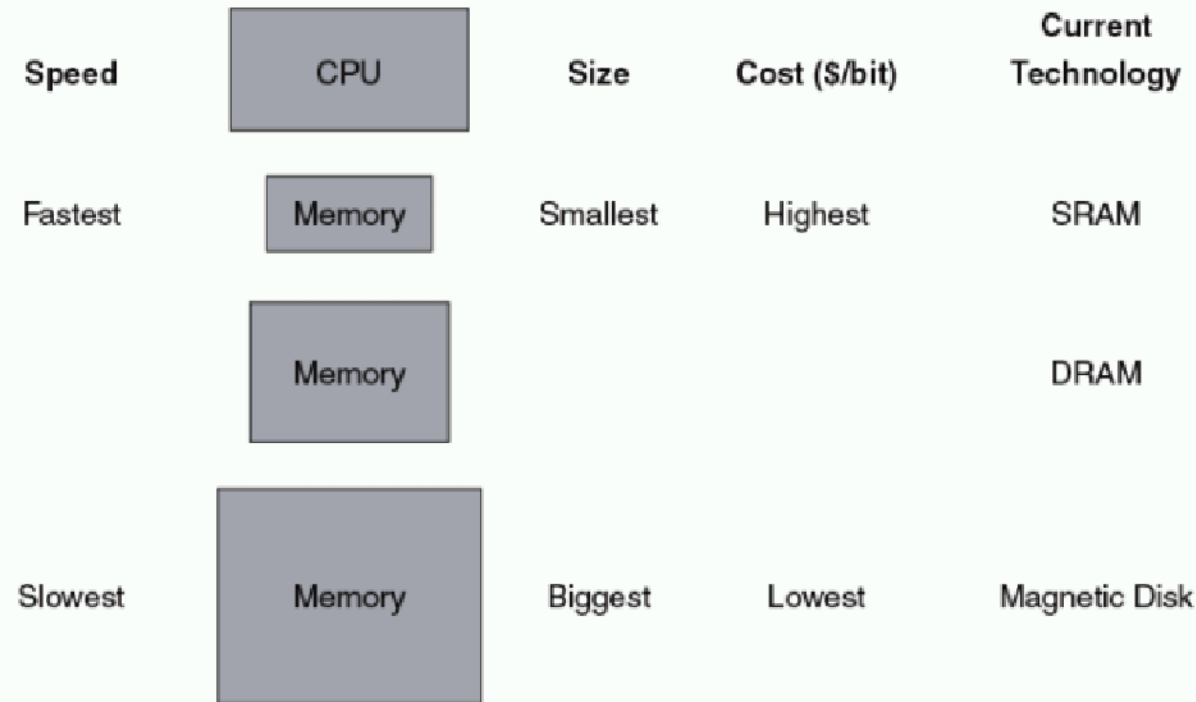
- Optical

CD

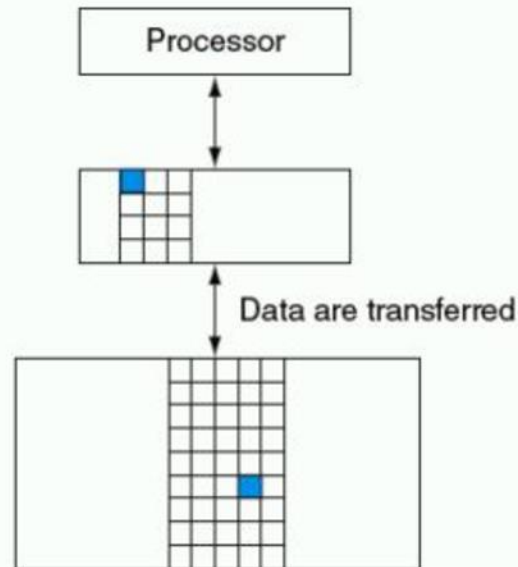
DVD

# Memory Hierarchy Analysis

- Memory
- Capacity
- Unit cost
- Total cost
- Access time
- Hit ratio
- Effective time



**FIGURE 7.1 The basic structure of a memory hierarchy.** By implementing the memory system as a hierarchy, the user has the illusion of a memory that is as large as the largest level of the hierarchy, but can be accessed as if it were all built from the fastest memory.



**FIGURE 7.2** Every pair of levels in the memory hierarchy can be thought of as having an **upper and lower level**. Within each level, the unit of information that is present or not is called a *block*. Usually we transfer an entire block when we copy something between levels.

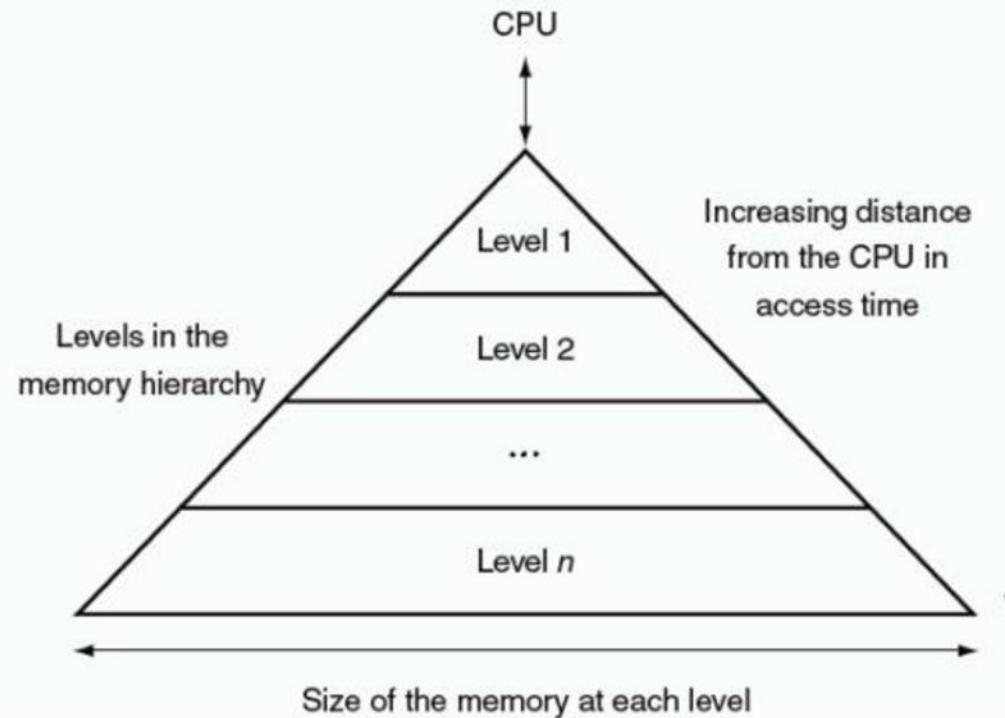
**block** The minimum unit of information that can be either present or not present in the two-level hierarchy.

**hit rate** The fraction of memory accesses found in a cache.

**miss rate** The fraction of memory accesses not found in a level of the memory hierarchy.

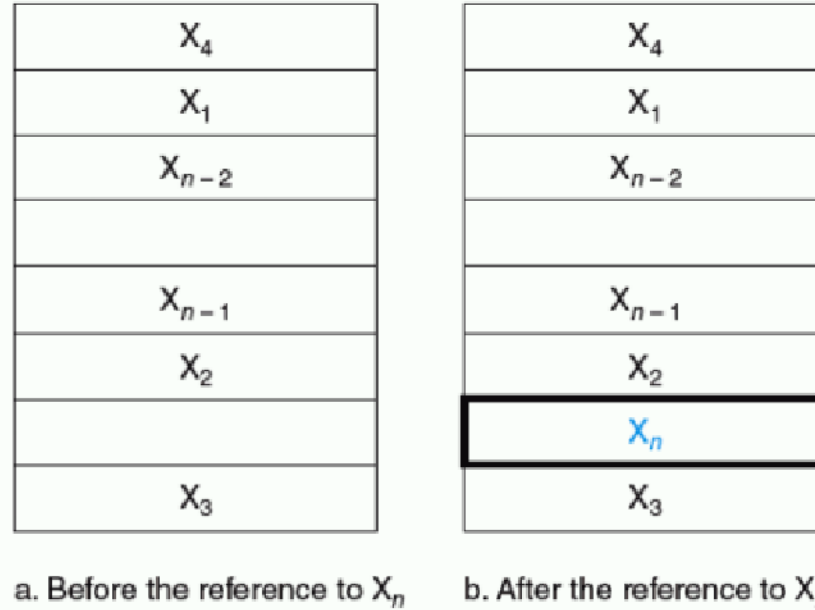
**hit time** The time required to access a level of the memory hierarchy, including the time needed to determine whether the access is a hit or a miss.

**miss penalty** The time required to fetch a block into a level of the memory hierarchy from the lower level, including the time to access the block, transmit it from one level to the other, and insert it in the level that experienced the miss.



**FIGURE 7.3 This diagram shows the structure of a memory hierarchy: as the distance from the processor increases, so does the size.** This structure with the appropriate operating mechanisms allows the processor to have an access time that is determined primarily by level 1 of the hierarchy and yet have a memory as large as level  $n$ . Maintaining this illusion is the subject of this chapter. Although the local disk is normally the bottom of the hierarchy, some systems use tape or a file server over a local area network as the next levels of the hierarchy.

# Cache memory



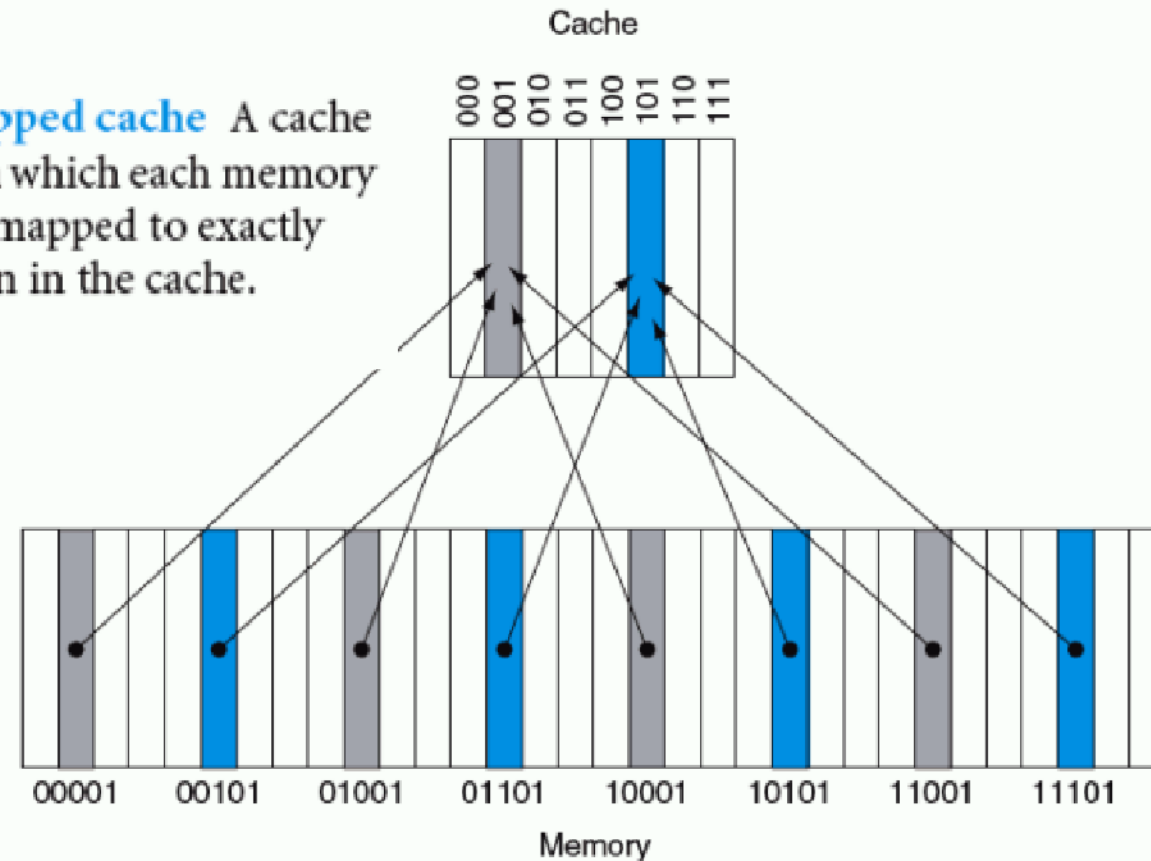
**FIGURE 7.4** The cache just before and just after a reference to a word  $X_n$  that is not initially in the cache. This reference causes a miss that forces the cache to fetch  $X_n$  from memory and insert it into the cache.

- How do you know if the data item is in the cache?
- If it is how do we find it?
- If each word can go in exactly one place in the cache, then it is straightforward to find the word if it is in the cache.
- The simplest way to assign a location in the cache for each word in the memory is to assign the cache location based on the address of the word in the memory. This cache structure is called **direct mapped**, since each memory location is mapped directly to exactly one location in the cache



# Direct-Mapped Cache

**direct-mapped cache** A cache structure in which each memory location is mapped to exactly one location in the cache.



**FIGURE 7.5** A direct-mapped cache with eight entries showing the addresses of memory words between 0 and 31 that map to the same cache locations. Because there are eight words in the cache, an address  $X$  maps to the cache word  $X \bmod 8$ . That is, the low-order  $\log_2(8) = 3$  bits are used as the cache index. Thus, addresses  $00001_{\text{two}}$ ,  $01001_{\text{two}}$ ,  $10001_{\text{two}}$ , and  $11001_{\text{two}}$  all map to entry  $001_{\text{two}}$  of the cache, while addresses  $00101_{\text{two}}$ ,  $01101_{\text{two}}$ ,  $10101_{\text{two}}$ , and  $11101_{\text{two}}$  all map to entry  $101_{\text{two}}$  of the cache.

- If the entries in the cache is a power of two, the modulo can be computed simply by using the lower order  $\log_2(\text{cache size in blocks})$  bits of the address; hence the cache may be accessed directly with the lower-order bits.
- Because each cache location can contain the contents of a number of different memory locations, how do we know whether the data in the cache corresponds to a requested word? (i.e. how do we know that the requested word is in cache or not?)
- Answer: adding a set of tags to the cache. The tags (upper portion of the address) contains the address information required to identify whether a word in the cache corresponds to the requested word.

- Valid bit: When a processor starts up the cache does not have good data and the tag fields will be meaningless.
- Even after executing many instructions, some of the cache entries may still be empty, thus the tag for such entries to be ignored.
- The most common method is to add a valid bit to indicate whether an entry contains a valid address.

# Accessing a cache

- When the word at the address 18(10010) is brought into the cache block 2 ,the word at 26(11010) replaced

| Decimal address of reference | Binary address of reference | Hit or miss in cache | Assigned cache block (where found or placed)                              |
|------------------------------|-----------------------------|----------------------|---------------------------------------------------------------------------|
| 22                           | 10110 <sub>two</sub>        | miss (7.6b)          | $(10\underline{110}_{\text{two}} \bmod 8) = \underline{110}_{\text{two}}$ |
| 26                           | 11010 <sub>two</sub>        | miss (7.6c)          | $(11\underline{010}_{\text{two}} \bmod 8) = \underline{010}_{\text{two}}$ |
| 22                           | 10110 <sub>two</sub>        | hit                  | $(10\underline{110}_{\text{two}} \bmod 8) = \underline{110}_{\text{two}}$ |
| 26                           | 11010 <sub>two</sub>        | hit                  | $(11\underline{010}_{\text{two}} \bmod 8) = \underline{010}_{\text{two}}$ |
| 16                           | 10000 <sub>two</sub>        | miss (7.6d)          | $(10\underline{000}_{\text{two}} \bmod 8) = \underline{000}_{\text{two}}$ |
| 3                            | 00011 <sub>two</sub>        | miss (7.6e)          | $(00\underline{011}_{\text{two}} \bmod 8) = \underline{011}_{\text{two}}$ |
| 16                           | 10000 <sub>two</sub>        | hit                  | $(10\underline{000}_{\text{two}} \bmod 8) = \underline{000}_{\text{two}}$ |
| 18                           | 10010 <sub>two</sub>        | miss (7.6f)          | $(10\underline{010}_{\text{two}} \bmod 8) = \underline{010}_{\text{two}}$ |

| Index | V | Tag | Data |
|-------|---|-----|------|
| 000   | N |     |      |
| 001   | N |     |      |
| 010   | N |     |      |
| 011   | N |     |      |
| 100   | N |     |      |
| 101   | N |     |      |
| 110   | N |     |      |
| 111   | N |     |      |

a. The initial state of the cache after power-on

| Index | V | Tag        | Data                     |
|-------|---|------------|--------------------------|
| 000   | N |            |                          |
| 001   | N |            |                          |
| 010   | Y | $11_{two}$ | Memory ( $11010_{two}$ ) |
| 011   | N |            |                          |
| 100   | N |            |                          |
| 101   | N |            |                          |
| 110   | Y | $10_{two}$ | Memory ( $10110_{two}$ ) |
| 111   | N |            |                          |

c. After handling a miss of address ( $11010_{two}$ )

| Index | V | Tag        | Data                    |
|-------|---|------------|-------------------------|
| 000   | N |            |                         |
| 001   | N |            |                         |
| 010   | N |            |                         |
| 011   | N |            |                         |
| 100   | N |            |                         |
| 101   | N |            |                         |
| 110   | Y | $10_{two}$ | Memory( $10110_{two}$ ) |
| 111   | N |            |                         |

b. After handling a miss of address ( $10110_{two}$ )

| Index | V | Tag        | Data                     |
|-------|---|------------|--------------------------|
| 000   | Y | $10_{two}$ | Memory ( $10000_{two}$ ) |
| 001   | N |            |                          |
| 010   | Y | $11_{two}$ | Memory ( $11010_{two}$ ) |
| 011   | N |            |                          |
| 100   | N |            |                          |
| 101   | N |            |                          |
| 110   | Y | $10_{two}$ | Memory ( $10110_{two}$ ) |
| 111   | N |            |                          |

d. After handling a miss of address ( $10000_{two}$ )

| Index | V | Tag               | Data                            |
|-------|---|-------------------|---------------------------------|
| 000   | Y | $10_{\text{two}}$ | Memory ( $10000_{\text{two}}$ ) |
| 001   | N |                   |                                 |
| 010   | Y | $11_{\text{two}}$ | Memory ( $11010_{\text{two}}$ ) |
| 011   | Y | $00_{\text{two}}$ | Memory ( $00011_{\text{two}}$ ) |
| 100   | N |                   |                                 |
| 101   | N |                   |                                 |
| 110   | Y | $10_{\text{two}}$ | Memory ( $10110_{\text{two}}$ ) |
| 111   | N |                   |                                 |

e. After handling a miss of address ( $00011_{\text{two}}$ )

| Index | V | Tag               | Data                            |
|-------|---|-------------------|---------------------------------|
| 000   | Y | $10_{\text{two}}$ | Memory ( $10000_{\text{two}}$ ) |
| 001   | N |                   |                                 |
| 010   | Y | $10_{\text{two}}$ | Memory ( $10010_{\text{two}}$ ) |
| 011   | Y | $00_{\text{two}}$ | Memory ( $00011_{\text{two}}$ ) |
| 100   | N |                   |                                 |
| 101   | N |                   |                                 |
| 110   | Y | $10_{\text{two}}$ | Memory ( $10110_{\text{two}}$ ) |
| 111   | N |                   |                                 |

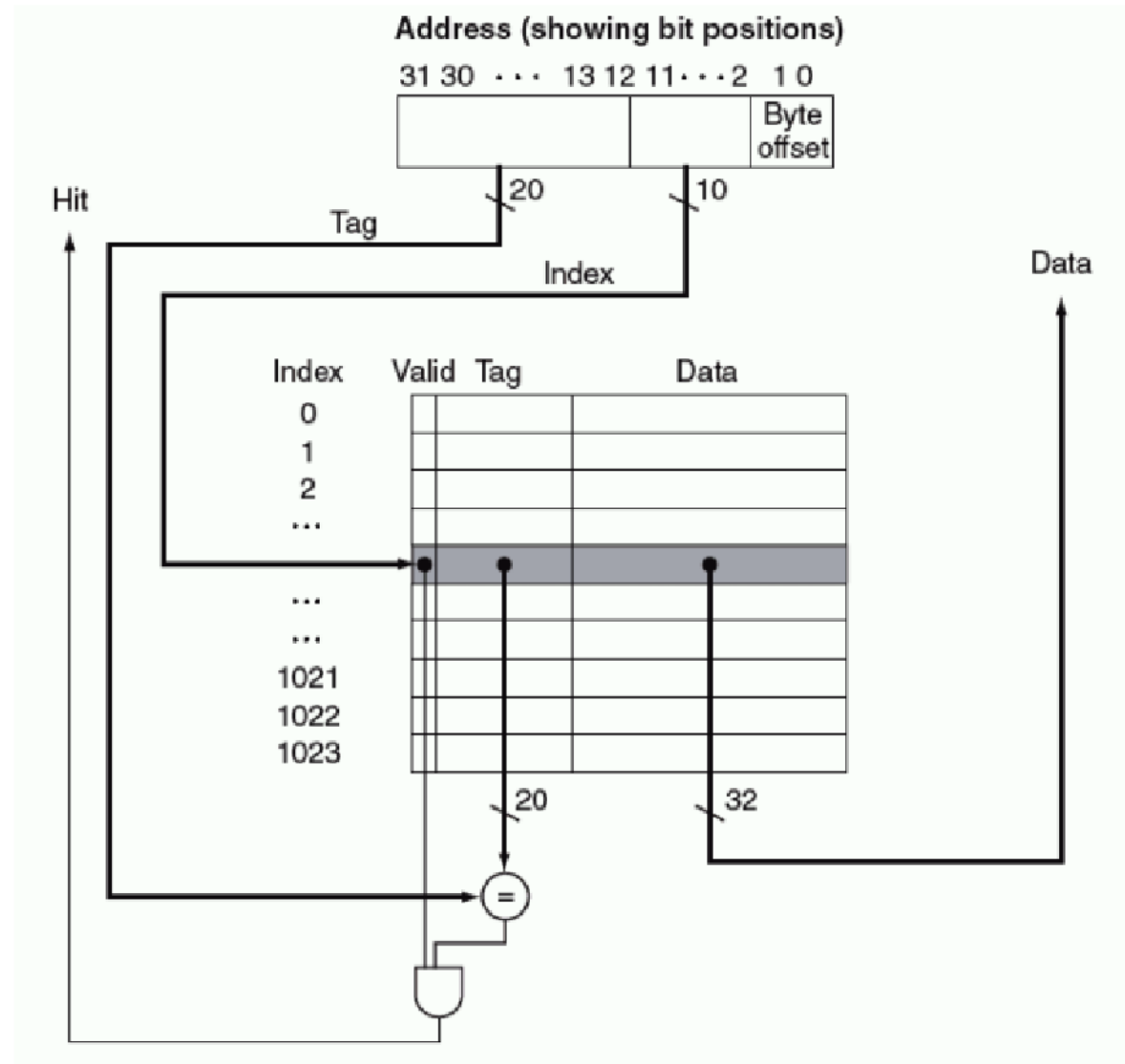
f. After handling a miss of address ( $10010_{\text{two}}$ )

**FIGURE 7.6 The cache contents are shown after each reference request that *misses*, with the index and tag fields shown in binary.** The cache is initially empty, with all valid bits (V entry in cache) turned off (N). The processor requests the following addresses:  $10110_{\text{two}}$  (miss),  $11010_{\text{two}}$  (miss),  $10110_{\text{two}}$  (hit),  $11010_{\text{two}}$  (hit),  $10000_{\text{two}}$  (miss),  $00011_{\text{two}}$  (miss),  $10000_{\text{two}}$  (hit), and  $10010_{\text{two}}$  (miss). The figures show the cache contents after each miss in the sequence has been handled. When address  $10010_{\text{two}}$  (18) is referenced, the entry for address  $11010_{\text{two}}$  (26) must be replaced, and a reference to  $11010_{\text{two}}$  will cause a subsequent miss. The tag field will contain only the upper portion of the address. The full address of a word contained in cache block  $i$  with tag field  $j$  for this cache is  $j \times 8 + i$ , or equivalently the concatenation of the tag field  $j$  and the index  $i$ . For example, in cache f above, index 010 has tag 10 and corresponds to address 10010.

All direct mapped caches use the  
mapping

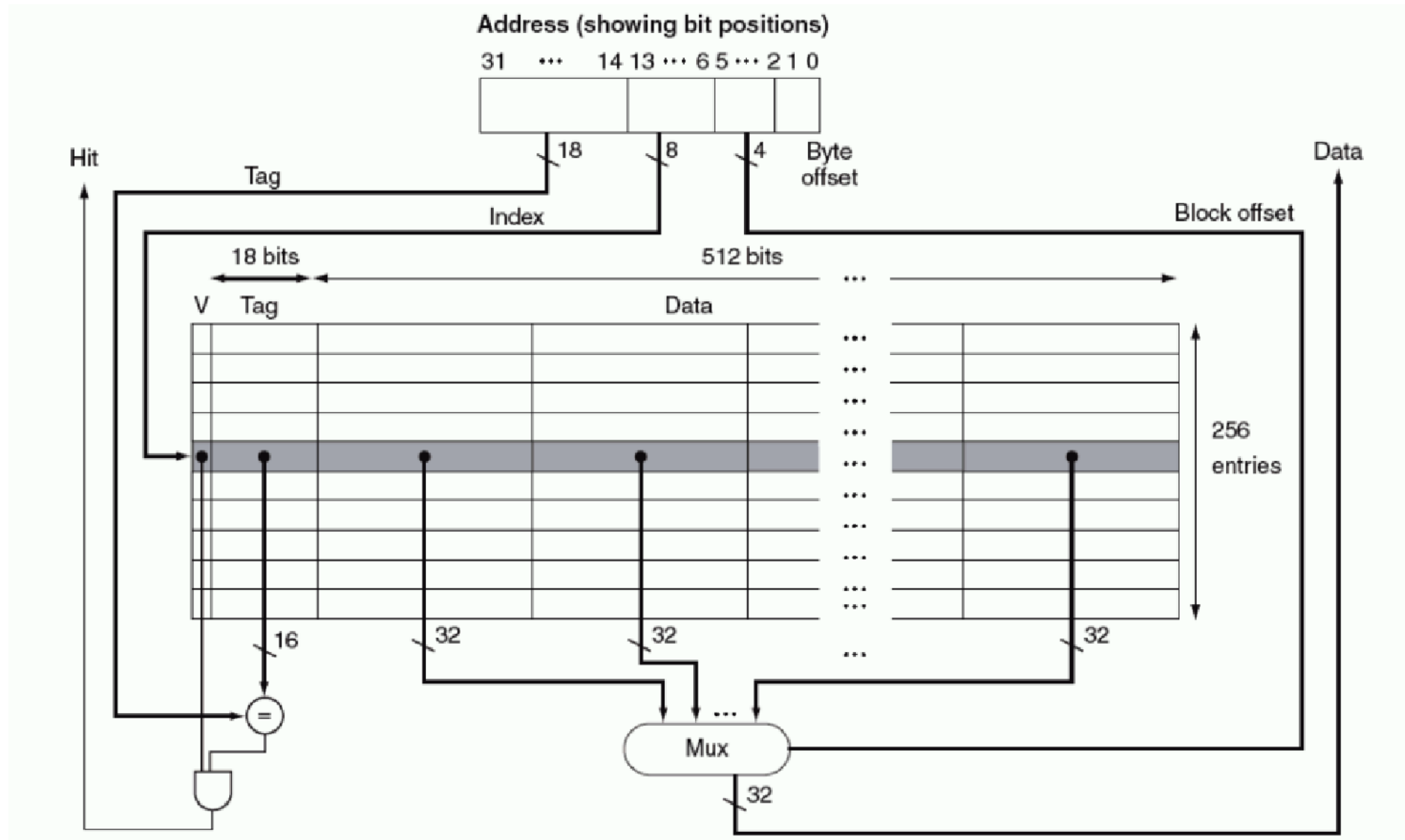
$(\text{Block address}) \bmod (\text{Number of cache blocks in the cache})$

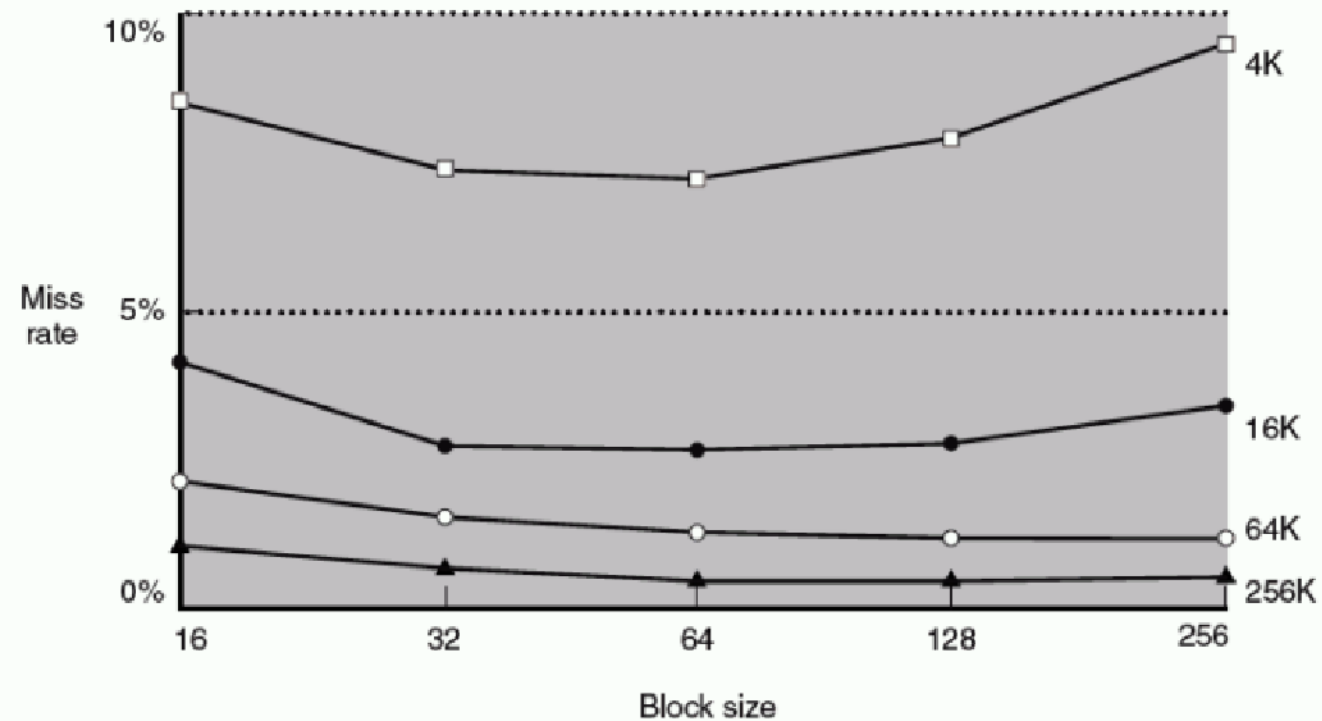
# Cache access mechanism





# Cache with four words





**FIGURE 7.8 Miss rate versus block size.** Note that miss rate actually goes up if the block size is too large relative to the cache size. Each line represents a cache of different size. (This figure is independent of associativity, discussed soon.) Unfortunately, SPEC2000 traces would take too long if block size were included, so these data are based on SPEC92.

# Direct-mapped cache

Main Memory

| Memory address | Data |
|----------------|------|
| 00000          | 1220 |
|                |      |
| 00777          | 2340 |
| 01000          | 3450 |
|                |      |
| 01777          | 4560 |
| 02000          | 5670 |
|                |      |
| 02777          | 6710 |

Cache Memory

| Index address | Tag | Data |
|---------------|-----|------|
| 000           | 00  | 1220 |
|               |     |      |
| 777           | 02  | 6710 |

- The disadvantage of direct mapped cache is that the hit ratio can drop considerably if two or more words whose addresses have the same index but different tags are accessed repeatedly. But this possibility is minimized by the fact that such words are relatively far apart in the address range.

# direct mapped cache with block size of 8 words(512 word cache)

|          | index | tag | data |
|----------|-------|-----|------|
| block0   | 000   | 01  | 3450 |
|          | 007   | 01  | 6578 |
| block1   | 010   |     |      |
|          | 017   |     |      |
|          | .     |     |      |
| block 63 | 770   | 02  |      |
|          | 777   | 02  | 6710 |

| Tag | Block | word |
|-----|-------|------|
|-----|-------|------|

index

- Disadvantage of direct mapping is that two words with the same index in their address but with different tags cannot reside in cache memory at the same time.
- Improvement :Set Associative Mapping

# Two-way Set Associative Mapping

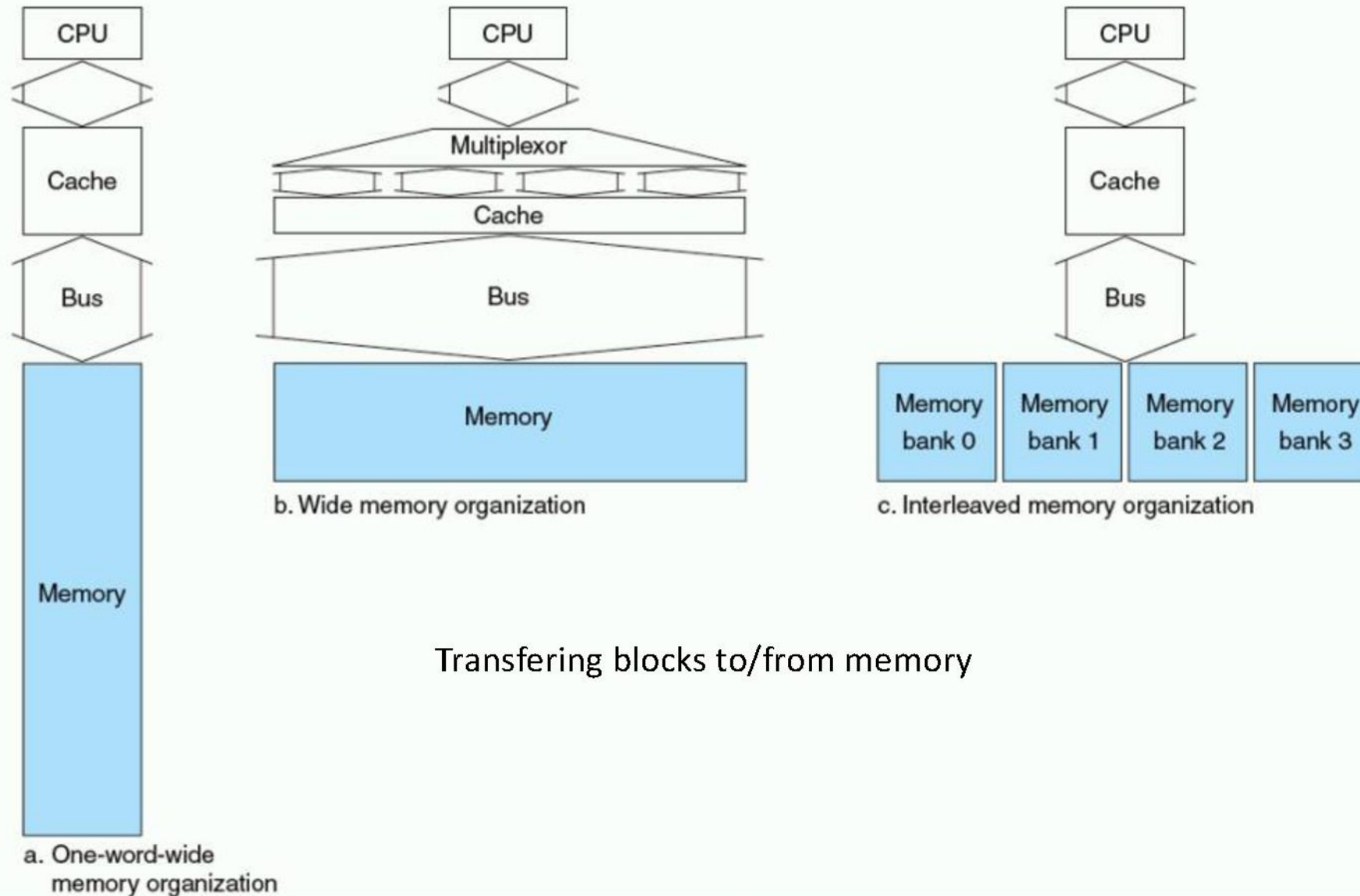
| Index | Tag | Data |  | Tag | Data |
|-------|-----|------|--|-----|------|
| 000   | 01  | 3450 |  | 02  | 5670 |
|       |     |      |  |     |      |
| 777   | 02  | 6710 |  | 00  | 2340 |

- The hit ratio will improve as the set size increases because more words with same index but different tags can reside in cache. An increase in the set size increases the number of bits in words of cache and requires more complex comparison logic.
- For replacement when miss occurs uses FIFO, LRU, random replacement algorithms.



How many total bits are required for a direct-mapped cache with 16 KB of data and 4-word blocks, assuming a 32-bit address?

**FIGURE 7.11 The primary method of achieving higher memory bandwidth is to increase the physical or logical width of the memory system.** In this figure, memory bandwidth is improved two ways. The simplest design, (a), uses a memory where all components are one word wide; (b) shows a wider memory, bus, and cache; while (c) shows a narrow bus and cache with an interleaved memory. In (b), the logic between the cache and processor consists of a multiplexor used on reads and control logic to update the appropriate words of the cache on writes.



# Designing the memory system to support the cache

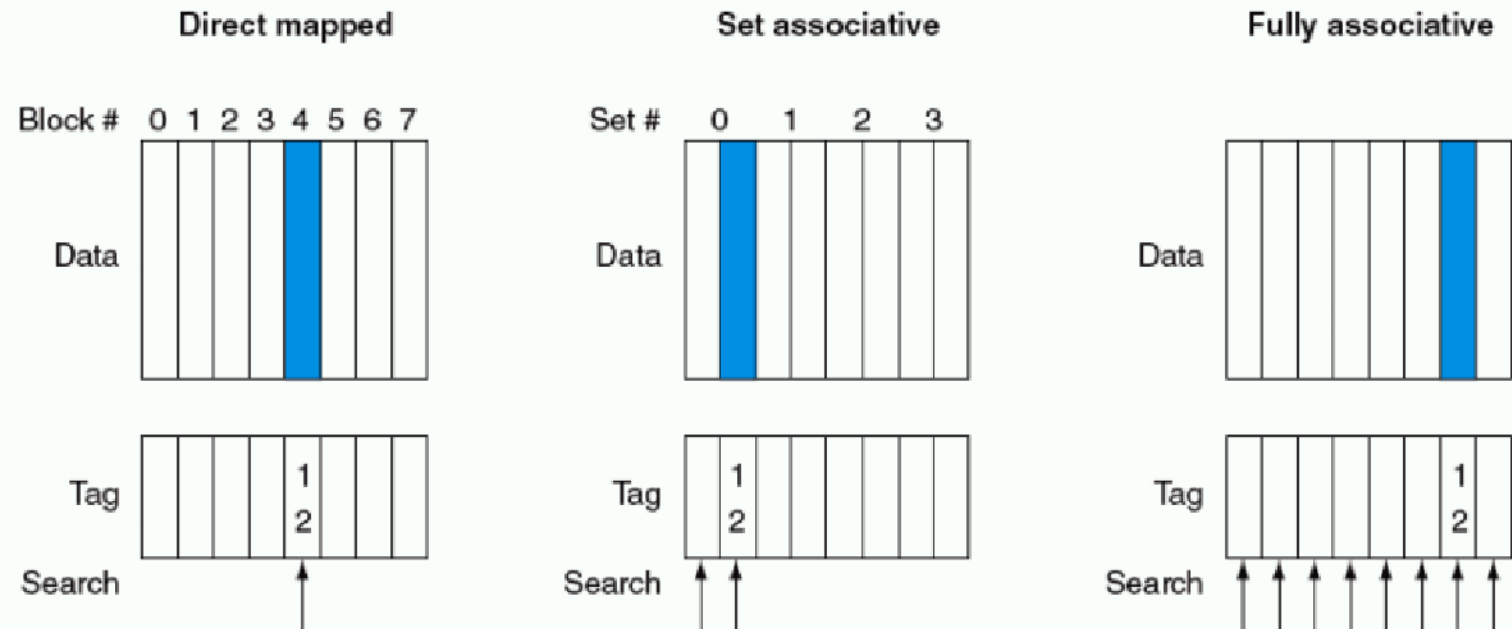
The processor is typically connected to memory over a bus. The clock rate of the bus is usually much slower than the processor, by as much as a factor of 10. The speed of this bus affects the miss penalty.

To understand the impact of different organizations of memory, let's define a set of hypothetical memory access times. Assume

- 1 memory bus clock cycle to send the address
- 15 memory bus clock cycles for each DRAM access initiated
- 1 memory bus clock cycle to send a word of data

If we have a cache block of four words and a one-word-wide bank of DRAMs, the miss penalty would be  $1 + 4 \times 15 + 4 \times 1 = 65$  memory bus clock cycles. Thus, the number of bytes transferred per bus clock cycle for a single miss would be

$$\frac{4 \times 4}{65} = 0.25$$



**FIGURE 7.13** The location of a memory block whose address is 12 in a cache with 8 blocks varies for direct-mapped, set-associative, and fully associative placement. In direct-mapped placement, there is only one cache block where memory block 12 can be found, and that block is given by  $(12 \bmod 8) = 4$ . In a two-way set-associative cache, there would be four sets, and memory block 12 must be in set  $(12 \bmod 4) = 0$ ; the memory block could be in either element of the set. In a fully associative placement, the memory block for block address 12 can appear in any of the eight cache blocks.

**One-way set associative  
(direct mapped)**

| Block | Tag | Data |
|-------|-----|------|
| 0     |     |      |
| 1     |     |      |
| 2     |     |      |
| 3     |     |      |
| 4     |     |      |
| 5     |     |      |
| 6     |     |      |
| 7     |     |      |

### Two-way set associative

| Set | Tag | Data | Tag | Data |
|-----|-----|------|-----|------|
| 0   |     |      |     |      |
| 1   |     |      |     |      |
| 2   |     |      |     |      |
| 3   |     |      |     |      |

### Four-way set associative

| Set | Tag | Data | Tag | Data | Tag | Data | Tag | Data |
|-----|-----|------|-----|------|-----|------|-----|------|
| 0   |     |      |     |      |     |      |     |      |
| 1   |     |      |     |      |     |      |     |      |

**Eight-way set associative (fully associative)**

[illegible]

# Intrinsity FastMATH Processor

- Intrinsity FastMATH Processor: It is a fast embedded microprocessor that uses the MIPS architecture and a simple cache implementation.
- This processor has a 12 stage pipeline. when operating at peak speed the processor can request both an instruction word and data on every clock cycle. To satisfy the demands of the pipeline without stalling, separate instruction and data caches are used. Each cache is 16KB, or 4K words, with 16 word blocks



Read requests for the cache are straightforward. Because there are separate data and instruction caches, separate control signals will be needed to read and write each cache. (Remember that we need to update the instruction cache when a miss occurs.) Thus, the steps for a read request to either cache are as follows:

1. Send the address to the appropriate cache. The address comes either from the PC (for an instruction) or from the ALU (for data).
2. If the cache signals hit, the requested word is available on the data lines. Since there are 16 words in the desired block, we need to select the right one. A block index field is used to control the multiplexor (shown at the bottom of the figure), which selects the requested word from the 16 words in the indexed block.
3. If the cache signals miss, we send the address to the main memory. When the memory returns with the data, we write it into the cache and then read it to fulfill the request.



For writes, the Intrinsity FastMATH offers both write-through and write-back, leaving it up to the operating system to decide which strategy to use for an application. It has a one-entry write buffer.

| Instruction miss rate | Data miss rate | Effective combined miss rate |
|-----------------------|----------------|------------------------------|
| 0.4%                  | 11.4%          | 3.2%                         |

**FIGURE 7.10 Approximate instruction and data miss rates for the Intrinsity FastMATH processor for SPEC2000 benchmarks.** The combined miss rate is the effective miss rate seen for the combination of the 16 KB instruction cache and 16 KB data cache. It is obtained by weighting the instruction and data individual miss rates by the frequency of instruction and data references.

**split cache** A scheme in which a level of the memory hierarchy is composed of two independent caches that operate in parallel with each other with one handling instructions and one handling data.

# Measuring and improving the cache performance

- Two different techniques for improving cache performance:

1) Reduce the miss rate by reducing the probability that two different memory blocks will contend for the same cache location.

2) Reduce the miss penalty by adding an additional level to the hierarchy.(multilevel caching)

CPU time can be divided into the clock cycles that the CPU spends executing the program and the clock cycles that the CPU spends waiting for the memory system. Normally, we assume that the costs of cache accesses that are hits are part of the normal CPU execution cycles. Thus,

$$\text{CPU time} = (\text{CPU execution clock cycles} + \text{Memory-stall clock cycles}) \\ \times \text{Clock cycle time}$$

The memory-stall clock cycles come primarily from cache misses,

Memory-stall clock cycles can be defined as the sum of the stall cycles coming from reads plus those coming from writes:

$$\text{Memory-stall clock cycles} = \text{Read-stall cycles} + \text{Write-stall cycles}$$

The read-stall cycles can be defined in terms of the number of read accesses per program, the miss penalty in clock cycles for a read, and the read miss rate:

$$\text{Read-stall cycles} = \frac{\text{Reads}}{\text{Program}} \times \text{Read miss rate} \times \text{Read miss penalty}$$

$$\begin{aligned} \text{Write-stall cycles} = & \left( \frac{\text{Writes}}{\text{Program}} \times \text{Write miss rate} \times \text{Write miss penalty} \right) \\ & + \text{Write buffer stalls} \end{aligned}$$

In most write-through cache organizations, the read and write miss penalties are the same (the time to fetch the block from memory). If we assume that the write buffer stalls are negligible, we can combine the reads and writes by using a single miss rate and the miss penalty:

$$\text{Memory-stall clock cycles} = \frac{\text{Memory accesses}}{\text{Program}} \times \text{Miss rate} \times \text{Miss penalty}$$

We can also factor this as

$$\text{Memory-stall clock cycles} = \frac{\text{Instructions}}{\text{Program}} \times \frac{\text{Misses}}{\text{Instruction}} \times \text{Miss penalty}$$

- A computer system has an L1 cache ,an L2 cache, and a main memory unit connected and at time 4 words can be transferred through data bus.The block size in L1 cache is 4 words .The block size in L2 cache is 16 words. The memory access times are 2 ns, 20ns and 200 ns for L1 ,L2 and main memory unit respectively.
- i)When there is a miss in L1 cache and a hit in L2 cache,a block is transferred from L2 cache to L1 cache. What is the time taken for this transfer?
- ii)When there is a miss in both L1 and L2,first a block is transferred from main memory to L2 and then L2 to L1.What is the total time taken for these transfers?

- A computer has a memory unit of  $64K \times 16$  and a cache memory of  $1K$  words. The cache uses direct mapping with a block size of four words. The number of bits in the tag, index, block, word field and number of blocks is respectively:

Assume there are three small caches, each consisting of four one-word blocks. One cache is fully associative, a second is two-way set associative, and the third is direct mapped. Find the number of misses for each cache organization given the following sequence of block addresses: 0, 8, 0, 6, 8.



# Calculating Cache Performance

Assume an instruction cache miss rate for a program is 2% and a data cache miss rate is 4%. If a processor has a CPI of 2 without any memory stalls and the miss penalty is 100 cycles for all misses, determine how much faster a processor would run with a perfect cache that never missed. Use the instruction frequency of all loads and stores in SPECint2000 is 36%.

The number of memory miss cycles for instructions in terms of the Instruction count (I) is

$$\text{Instruction miss cycles} = I \times 2\% \times 100 = 2.00 \times I$$

The frequency of all loads and stores in SPECint2000 is 36%. Therefore, we can find the number of memory miss cycles for data references:

$$\text{Data miss cycles} = I \times 36\% \times 4\% \times 100 = 1.44 \times I$$

The total number of memory-stall cycles is  $2.00 I + 1.44 I = 3.44 I$ . This is more than 3 cycles of memory stall per instruction. Accordingly, the CPI with memory stalls is  $2 + 3.44 = 5.44$ . Since there is no change in instruction count or clock rate, the ratio of the CPU execution times is

$$\begin{aligned} \frac{\text{CPU time with stalls}}{\text{CPU time with perfect cache}} &= \frac{I \times \text{CPI}_{\text{stall}} \times \text{Clock cycle}}{I \times \text{CPI}_{\text{perfect}} \times \text{Clock cycle}} \\ &= \frac{\text{CPI}_{\text{stall}}}{\text{CPI}_{\text{perfect}}} = \frac{5.44}{2} \end{aligned}$$

The performance with the perfect cache is better by  $\frac{5.44}{2} = 2.72$ .

- Let us Speed up the computer by reducing CPI from 2 to 1 without changing the clock rate, which might be done with improved pipeline:

The system with cache misses would then have a CPI of  $1 + 3.44 = 4.44$ , and the system with the perfect cache would be

$$\frac{4.44}{1} = 4.44 \text{ times faster}$$

The amount of execution time spent on memory stalls would have risen from

$$\frac{3.44}{5.44} = 63\%$$

to

$$\frac{3.44}{4.44} = 77\%$$

## Cache performance with increased clock rate

Suppose we increase the performance of the computer in the previous example by doubling its clock rate. Since the main memory speed is unlikely to change, assume that the absolute time to handle a cache miss does not change. How much faster will the computer be with the faster clock, assuming the same miss rate as the previous example?

Measured in the faster clock cycles, the new miss penalty will be twice as many clock cycles, or 200 clock cycles. Hence:

$$\text{Total miss cycles per instruction} = (2\% \times 200) + 36\% \times (4\% \times 200) = 6.88$$

Thus, the faster computer with cache misses will have a CPI of  $2 + 6.88 = 8.88$ , compared to a CPI with cache misses of 5.44 for the slower computer.

Using the formula for CPU time from the previous example, we can compute the relative performance as

$$\frac{\text{Performance with fast clock}}{\text{Performance with slow clock}} = \frac{\text{Execution time with slow clock}}{\text{Execution time with fast clock}}$$

$$= \frac{\text{IC} \times \text{CPI}_{\text{slow clock}} \times \text{Clock cycle}}{\text{IC} \times \text{CPI}_{\text{fast clock}} \times \frac{\text{Clock cycle}}{2}}$$

$$= \frac{5.44}{8.88 \times \frac{1}{2}} = 1.23$$

Thus, the computer with the faster clock is about 1.2 times faster rather than 2 times faster, which it would have been if we ignored cache misses.

1. The lower the CPI, the more pronounced the impact of stall cycles.
2. The main memory system is unlikely to improve as fast as processor cycle time, primarily because the performance of the underlying DRAM is not getting much faster. When calculating CPI, the cache miss penalty is measured in processor clock cycles needed for a miss. Therefore, if the main memories of two processors have the same absolute access times, a higher processor clock rate leads to a larger miss penalty.

Thus, the importance of cache performance for processors with low CPI and high clock rates is greater, and consequently the danger of neglecting cache behavior in assessing the performance of such processors is greater. 



| Block address | Cache block      |
|---------------|------------------|
| 0             | (0 modulo 4) = 0 |
| 6             | (6 modulo 4) = 2 |
| 8             | (8 modulo 4) = 0 |

| Address of memory block accessed | Hit or miss | Contents of cache blocks after reference |   |           |   |
|----------------------------------|-------------|------------------------------------------|---|-----------|---|
|                                  |             | 0                                        | 1 | 2         | 3 |
| 0                                | miss        | Memory[0]                                |   |           |   |
| 8                                | miss        | Memory[8]                                |   |           |   |
| 0                                | miss        | Memory[0]                                |   |           |   |
| 6                                | miss        | Memory[0]                                |   | Memory[6] |   |
| 8                                | miss        | Memory[8]                                |   | Memory[6] |   |

| Block address | Cache set        |
|---------------|------------------|
| 0             | (0 modulo 2) = 0 |
| 6             | (6 modulo 2) = 0 |
| 8             | (8 modulo 2) = 0 |

| Address of memory block accessed | Hit or miss | Contents of cache blocks after reference |           |       |       |
|----------------------------------|-------------|------------------------------------------|-----------|-------|-------|
|                                  |             | Set 0                                    | Set 0     | Set 1 | Set 1 |
| 0                                | miss        | Memory[0]                                |           |       |       |
| 8                                | miss        | Memory[0]                                | Memory[8] |       |       |
| 0                                | hit         | Memory[0]                                | Memory[8] |       |       |
| 6                                | miss        | Memory[0]                                | Memory[6] |       |       |
| 8                                | miss        | Memory[8]                                | Memory[6] |       |       |

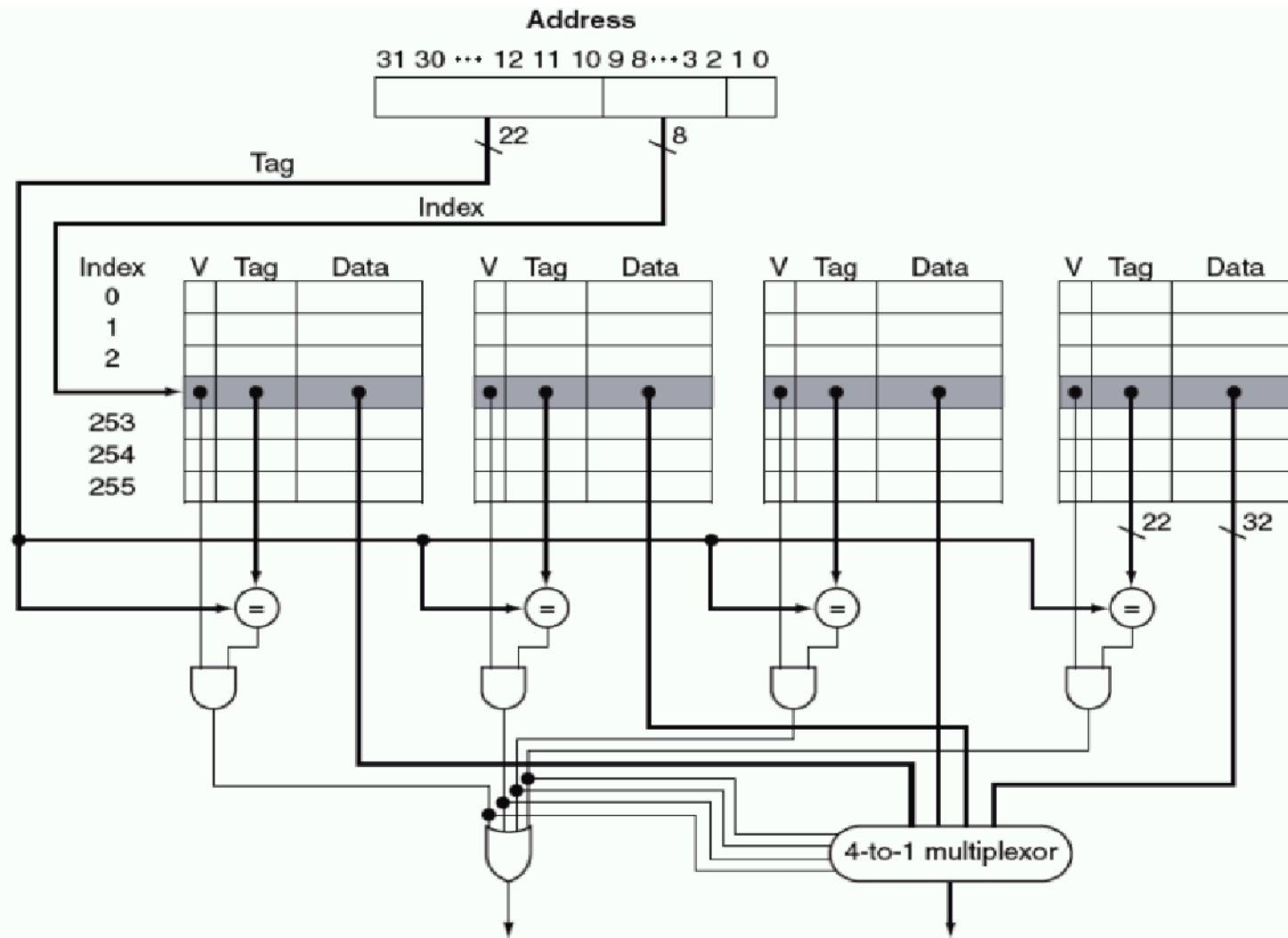
| Address of memory block accessed | Hit or miss | Contents of cache blocks after reference |           |           |         |
|----------------------------------|-------------|------------------------------------------|-----------|-----------|---------|
|                                  |             | Block 0                                  | Block 1   | Block 2   | Block 3 |
| 0                                | miss        | Memory[0]                                |           |           |         |
| 8                                | miss        | Memory[0]                                | Memory[8] |           |         |
| 0                                | hit         | Memory[0]                                | Memory[8] |           |         |
| 6                                | miss        | Memory[0]                                | Memory[8] | Memory[6] |         |
| 8                                | hit         | Memory[0]                                | Memory[8] | Memory[6] |         |

| Associativity | Data miss rate |
|---------------|----------------|
| 1             | 10.3%          |
| 2             | 8.6%           |
| 4             | 8.3%           |
| 8             | 8.1%           |

**FIGURE 7.15** The data cache miss rates for an organization like the Intrinsity FastMATH processor for SPEC2000 benchmarks with associativity varying from one-way to eight-way. These results for 10 SPEC2000 programs are from Hennessy and Patterson [2003].



## Locating block in a cache



The choice among direct-mapped, set-associative, or fully associative mapping in any memory hierarchy will depend on the cost of a miss versus the cost of implementing associativity, both in time and in extra hardware.

# Sizes of tags vs set associativity

Increasing associativity requires more comparators, and more tag bits per cache block. Assuming a cache of 4K blocks, a four-word block size, and a 32-bit address, find the total number of sets and the total number of tag bits for caches that are direct mapped, two-way and four-way set associative, and fully associative.

Since there are 16 ( $=2^4$ ) bytes per block, a 32-bit address yields  $32 - 4 = 28$  bits to be used for index and tag. The direct-mapped cache has the same number of sets as blocks, and hence 12 bits of index, since  $\log_2(4K) = 12$ ; hence, the total number of tag bits is  $(28 - 12) \times 4K = 16 \times 4K = 64$  Kbits.

Each degree of associativity decreases the number of sets by a factor of two and thus decreases the number of bits used to index the cache by one and increases the number of bits in the tag by one. Thus, for a two-way set-associative cache, there are  $2K$  sets, and the total number of tag bits is  $(28 - 11) \times 2 \times 2K = 34 \times 2K = 68$  Kbits. For a four-way set-associative cache, the total number of sets is  $1K$ , and the total number of tag bits is  $(28 - 10) \times 4 \times 1K = 72 \times 1K = 72$  Kbits.

For a fully associative cache, there is only one set with  $4K$  blocks, and the tag is 28 bits, leading to a total of  $28 \times 4K \times 1 = 112K$  tag bits.

# Choosing which block to replace

When a miss occurs in a direct-mapped cache, the requested block can go in exactly one position, and the block occupying that position must be replaced. In an associative cache, we have a choice of where to place the requested block, and hence a choice of which block to replace. In a fully associative cache, all blocks are candidates for replacement. In a set-associative cache, we must choose among the blocks in the selected set.

- Commonly used scheme:LRU
- As associativity increases ,implementing LRU gets harder

# Performance of multilevel caches

Suppose we have a processor with a base CPI of 1.0, assuming all references hit in the primary cache, and a clock rate of 5 GHz. Assume a main memory access time of 100 ns, including all the miss handling. Suppose the miss rate per instruction at the primary cache is 2%. How much faster will the processor be if we add a secondary cache that has a 5 ns access time for either a hit or a miss and is large enough to reduce the miss rate to main memory to 0.5%?

The effective CPI with one level of caching is given by

$$\text{Total CPI} = \text{Base CPI} + \text{Memory-stall cycles per instruction}$$

For the processor with one level of caching,

$$\text{Total CPI} = 1.0 + \text{Memory-stall cycles per instruction} = 1.0 + 2\% \times 500 = 11.0$$

The miss penalty to main memory is

$$\frac{100 \text{ ns}}{0.2 \frac{\text{ns}}{\text{clock cycle}}} = 500 \text{ clock cycles}$$

With two levels of cache, a miss in the primary (or first-level) cache can be satisfied either by the secondary cache or by main memory. The miss penalty for an access to the second-level cache is

$$\frac{5 \text{ ns}}{0.2 \frac{\text{ns}}{\text{clock cycle}}} = 25 \text{ clock cycles}$$

If the miss is satisfied in the secondary cache, then this is the entire miss penalty. If the miss needs to go to main memory, then the total miss penalty is the sum of the secondary cache access time and the main memory access time.

Thus, for a two-level cache, total CPI is the sum of the stall cycles from both levels of cache and the base CPI:

$$\begin{aligned} \text{Total CPI} &= 1 + \text{Primary stalls per instruction} \\ &\quad + \text{Secondary stalls per instruction} \\ &= 1 + 2\% \times 25 + 0.5\% \times 500 = 1 + 0.5 + 2.5 = 4.0 \end{aligned}$$

Thus, the processor with the secondary cache is faster by

$$\frac{11.0}{4.0} = 2.8$$

Alternatively, we could have computed the stall cycles by summing the stall cycles of those references that hit in the secondary cache  $((2\% - 0.5\%) \times 25 = 0.4)$  and those references that go to main memory, which must include the cost to access the secondary cache as well as the main memory access time  $(0.5\% \times (25 + 500) = 2.6)$ . The sum,  $1.0 + 0.4 + 2.6$ , is again 4.0.